

---

---

# CSS Layout

— Flexbox and Grid —

---

---

# Hip & Happenin' Headline

## Contact

## Where It's At

Integer ornare neque enim, non imperdiet ex pellentesque sed. Sed congue, nunc ut rhoncus consectetur, diam purus venenatis lacus, vel sollicitudin nibh tortor sed nunc. Vestibulum at ante ac tortor ultricies gravida eget molestie nibh. Fusce tempor dictum lectus, id luctus sapien tristique suscipit. Cras volutpat lectus at urna vulputate pellentesque. Praesent eleifend, sapien ut finibus facilisis, orci augue elementum leo, a faucibus augue nibh ac urna. Aliquam erat volutpat. Sed ultrices tempus neque, nec iaculis orci sollicitudin id. Mauris gravida congue sapien, vitae finibus nisi condimentum id. Donec turpis metus, euismod sed luctus sit amet, semper eu purus.

## News

The small print

# Topics

|                             |                            |
|-----------------------------|----------------------------|
| Flex containers and items   | Grid containers and items  |
| Flow direction and wrapping | Setting up a grid template |
| Flex item alignment         | Placing items in the grid  |
| Controlling item “flex”     | Implicit grid features     |
|                             | Grid item alignment        |

# CSS Page Layout Tools

**Flexbox** for greater control over **arranging items along one axis**

**Grid** for two-dimensional **grid-based layouts**

You can use both together to achieve layouts

# Flexible Boxes with CSS Flexbox

The defining aspect of flex layout is the ability to make the flex items “**flex**,” altering their width/height to fill the available space in the main dimension

When you make an element a flexbox, all of its child elements line up on one axis

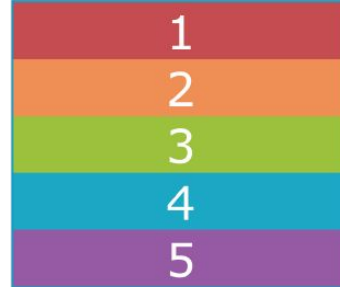
# Setting Up a Flexbox Container

To turn on flexbox mode for an element, set its **display** property to **flex** or **inline-flex**

```
#container {  
  display: flex;  
}
```

```
<div id="container">  
  <div class="box box1">1</div>  
  <div class="box box2">2</div>  
  <div class="box box3">3</div>  
  <div class="box box4">4</div>  
  <div class="box box5">5</div>  
</div>
```

By default, the `div`s display as block elements, stacking up vertically. Turning on flexbox mode makes them line up in a row.

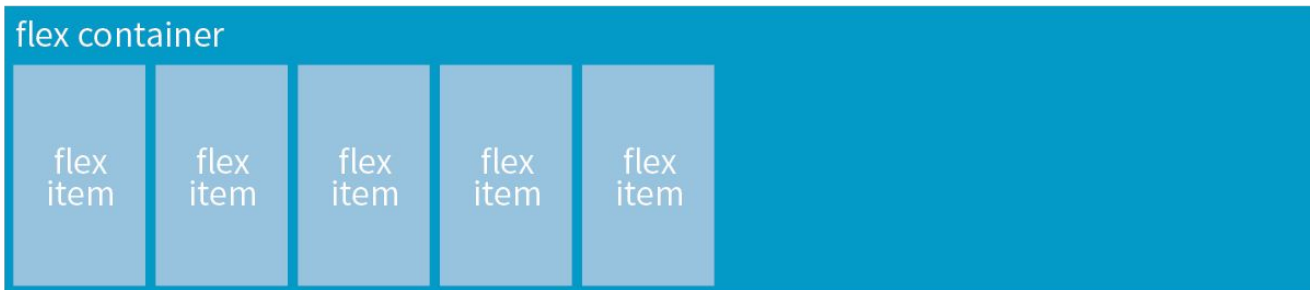


block layout mode

`display: flex;`



flexbox layout mode



# Setting Up a Flexbox Container

You can turn any flex item into a flex container by setting its **display** to **flex**, resulting in a nested flexbox

# Specifying flow direction

## **flex-direction**

Values: `row` | `column` | `row-reverse` | `column-reverse`

Default: `row`

flex-direction: **row**; (default)



flex-direction: **row-reverse**;



flex-direction: **column**;



flex-direction: **column-reverse**;



# Flex Terminology

Flexbox is **direction-agnostic**, there are no references to “left,” “right,” “top,” or “bottom” in the property values

We use terms **Main-Axis** and **Cross-Axis**

# Flex Terminology

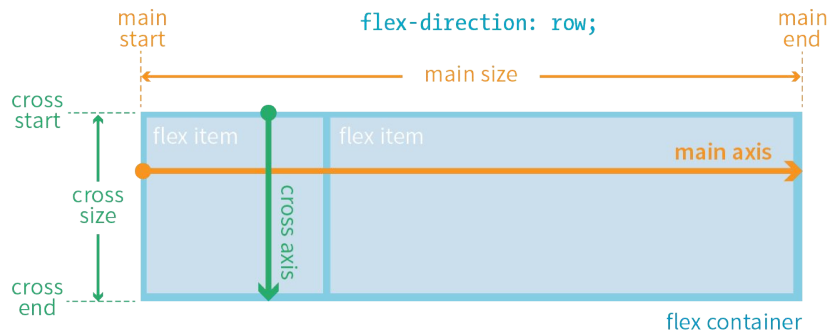
The **main axis** is the flow direction you've specified for the flex container

When **flex-direction** is set to **row**, the **main axis** is horizontal

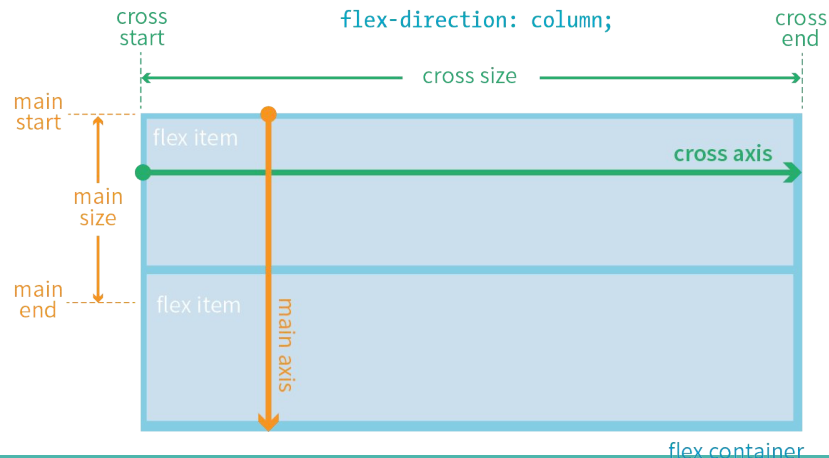
When **flex-direction** is set to **column**, the **main axis** is vertical

FOR LANGUAGES THAT READ HORIZONTALLY FROM LEFT TO RIGHT:

When **flex-direction** is set to **row**, the main axis is horizontal and the cross axis is vertical.



When **flex-direction** is set to **column**, the main axis is vertical and the cross axis is horizontal.



# Flex Terminology

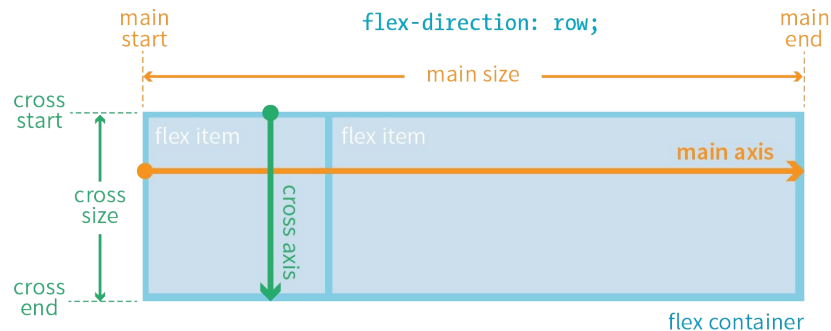
The **cross axis** is whatever direction is perpendicular to the main axis

vertical for row

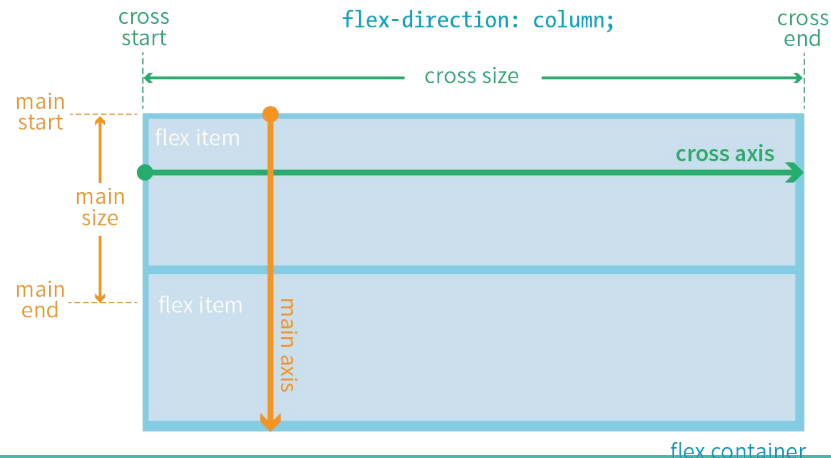
horizontal for column

FOR LANGUAGES THAT READ HORIZONTALLY FROM LEFT TO RIGHT:

When **flex-direction** is set to **row**, the main axis is horizontal and the cross axis is vertical.



When **flex-direction** is set to **column**, the main axis is vertical and the cross axis is horizontal.

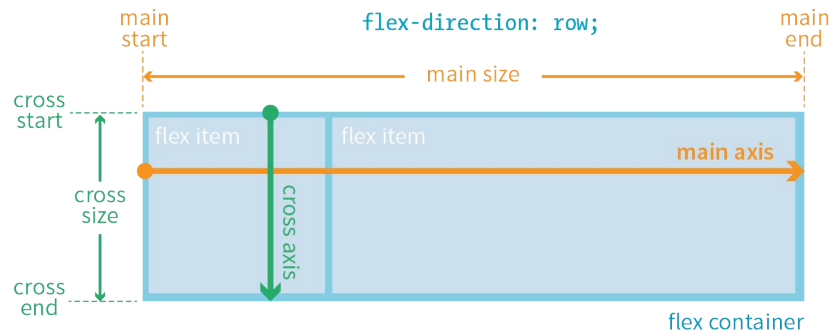


# Flex Terminology

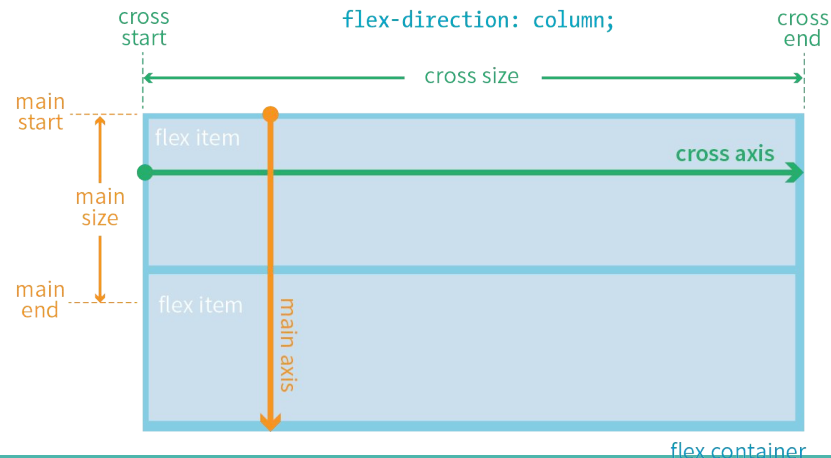
Both the main and cross axes have a **start** and an **end**, based on the direction in which the items flow

FOR LANGUAGES THAT READ HORIZONTALLY FROM LEFT TO RIGHT:

When `flex-direction` is set to `row`, the main axis is horizontal and the cross axis is vertical.



When `flex-direction` is set to `column`, the main axis is vertical and the cross axis is horizontal.



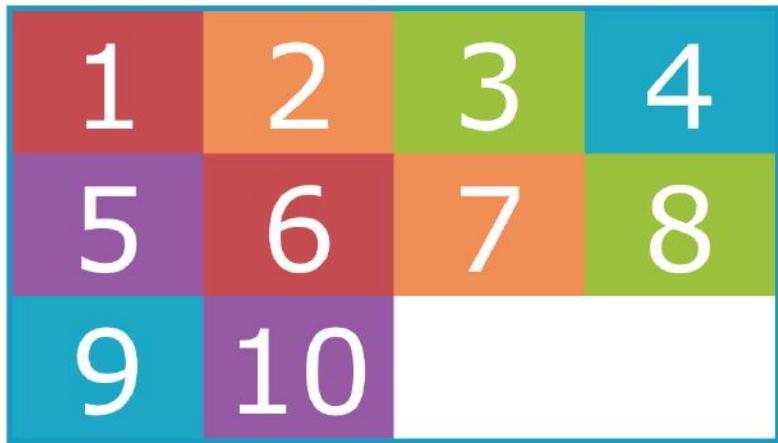
# Wrapping onto multiple lines

## **flex-wrap**

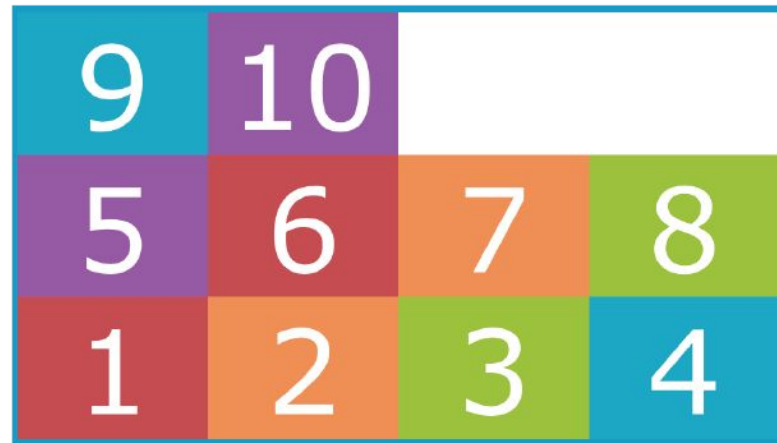
Values: nowrap | wrap | wrap-reverse

Default: nowrap

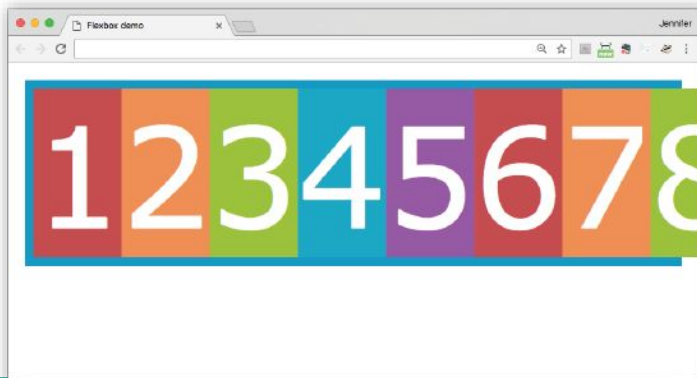
flex-wrap: **wrap**;



flex-wrap: **wrap-reverse**;

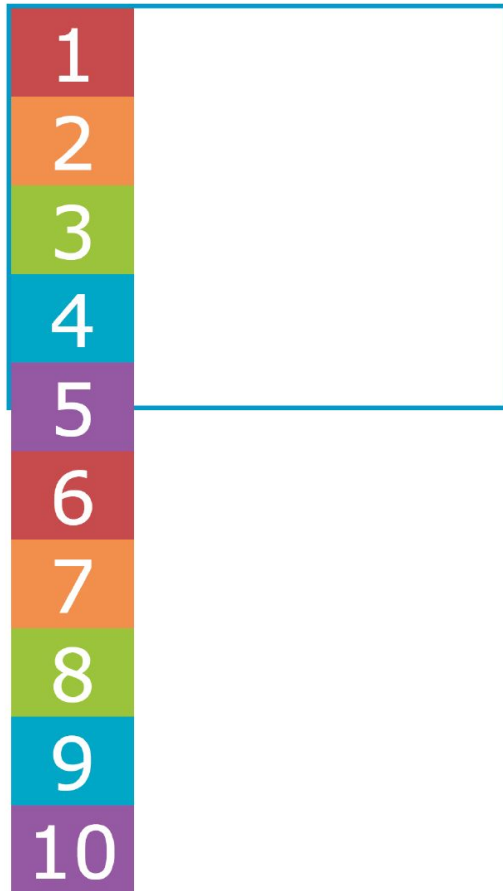


flex-wrap: **nowrap**; (default)

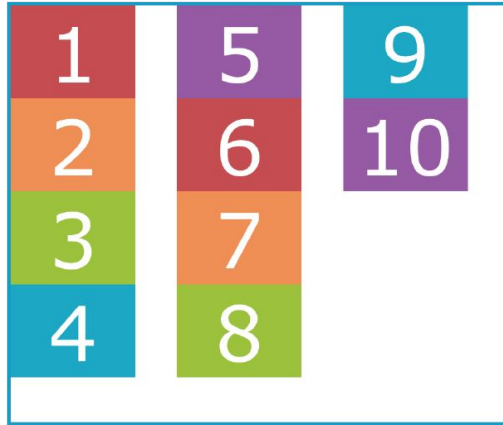


When wrapping is disabled, flex items squish if there is not enough room, and if they can't squish any further, may get cut off if there is not enough room in the viewport.

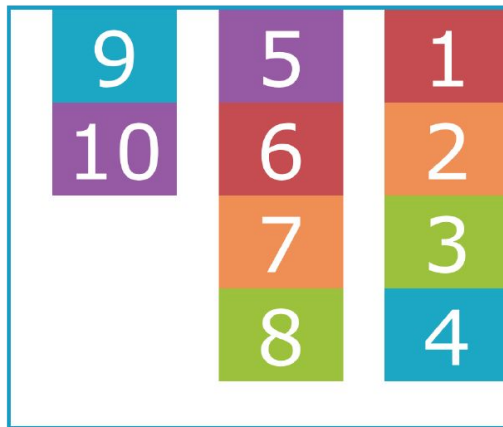
flex-wrap: nowrap; (default)



flex-wrap: wrap;



flex-wrap: wrap-reverse;



# Shorthand property `flex-flow`

## `flex-flow`

Values: `flex-direction` `flex-wrap`

Default: `row nowrap`

# Making a navigation bar with Flexbox



```
nav ul {  
  margin: 0;  
  padding: 0;  
  list-style-type: none;  
}
```

# Making a navigation bar with Flexbox



```
nav ul {  
  margin: 0;  
  padding: 0;  
  list-style-type: none;  
  display: flex;  
  justify-content: center;  
}
```

# Making a navigation bar with Flexbox



```
nav ul li a {  
  display: block;  
  border: 1px solid;  
  border-radius: 0.5em;  
  padding: 0.5em 1em;  
  margin: 0.5em;  
}
```

# Aligning Items on the main axis

## `justify-content`

Values: `flex-start` | `flex-end` | `center` | `space-between` | `space-around`

Default: `flex-start`

```
#container {  
  display: flex;  
  justify-content: flex-start;  
}
```

`justify-content: flex-start;` (default)



`justify-content: flex-end;`



`justify-content: center;`



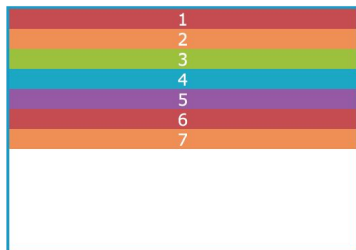
`justify-content: space-between;`



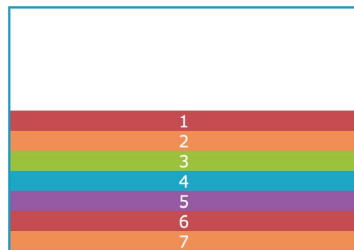
`justify-content: space-around;`



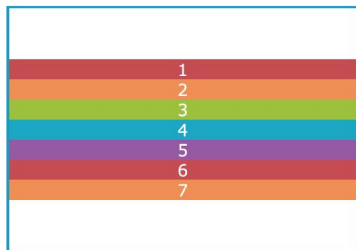
`justify-content: flex-start;`



`justify-content: flex-end;`



`justify-content: center;`



`justify-content: space-between;`



`justify-content: space-around;`



# Aligning on the cross axis

## `align-items`

Values: `flex-start` | `flex-end` | `center` | `baseline` | `stretch`

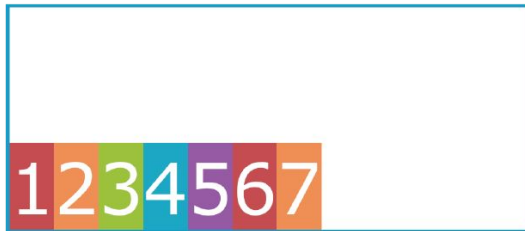
Default: `stretch`

```
#container {  
  display: flex;  
  flex-direction: row;  
  height: 200px;  
  align-items: flex-start;  
}
```

`align-items: flex-start;`



`align-items: flex-end;`



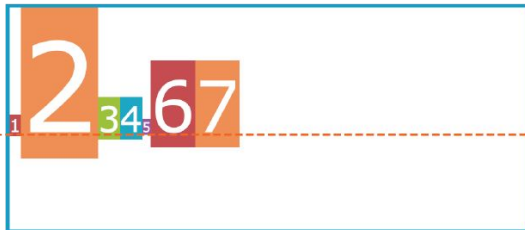
`align-items: center;`



`align-items: stretch;` (default)



`align-items: baseline;`



Items are aligned so that the baselines of the first text lines align.

# Aligning on the cross axis

If you'd like one or more items to override the cross-axis setting, use the **align-self** property on the individual item element(s)

## **align-self**

Values: flex-start | flex-end | center | baseline | stretch

Default: stretch

`align-self: flex-end;`



# Aligning multiple lines

## align-content

Values: flex-start | flex-end |  
center | space-around |  
space-between | stretch

Default: stretch

```
#container {  
  display: flex;  
  flex-direction: row;  
  flex-wrap: wrap;  
  height: 350px;  
  align-content: flex-start;  
}
```

`align-content: flex-start;`

|   |    |   |   |
|---|----|---|---|
| 1 | 2  | 3 | 4 |
| 5 | 6  | 7 | 8 |
| 9 | 10 |   |   |
|   |    |   |   |

`align-content: flex-end;`

|   |    |   |   |
|---|----|---|---|
| 1 | 2  | 3 | 4 |
| 5 | 6  | 7 | 8 |
| 9 | 10 |   |   |
|   |    |   |   |

`align-content: center;`

|   |    |   |   |
|---|----|---|---|
|   |    |   |   |
| 1 | 2  | 3 | 4 |
| 5 | 6  | 7 | 8 |
| 9 | 10 |   |   |
|   |    |   |   |

`align-content: space-between;`

|   |    |   |   |
|---|----|---|---|
| 1 | 2  | 3 | 4 |
| 5 | 6  | 7 | 8 |
| 9 | 10 |   |   |
|   |    |   |   |

`align-content: space-around;`

|   |    |   |   |
|---|----|---|---|
| 1 | 2  | 3 | 4 |
| 5 | 6  | 7 | 8 |
| 9 | 10 |   |   |
|   |    |   |   |

`align-content: stretch;` (default)

|   |    |   |   |
|---|----|---|---|
| 1 | 2  | 3 | 4 |
| 5 | 6  | 7 | 8 |
| 9 | 10 |   |   |
|   |    |   |   |

# Aligning multiple lines

The **align-content** property applies to the flex container element

A height is required for the container, because without it the container would be just tall enough to accommodate the content and there would be no space left over

The **align-content** property applies only when **flex-wrap** is set to **wrap** or **wrap-reverse** and there are multiple lines to align

# Determining How Items “Flex” in the Container

The concept of **flex** is concerned with **how space is distributed within items, growing or shrinking** items as required to make them fit

The **flex** property, which specifies how much an **item** can **grow** and **shrink**, and identifies its **starting size**

**flex** is a shorthand property for **flex-grow**, **flex-shrink**, and **flex-basis**

The **flex** properties apply to flex items, not the container

# Determining How Items “Flex” in the Container

## **flex**

Values: none | 'flex-grow flex-shrink flex-basis'

Default: 0 1 auto

# Determining How Items “Flex” in the Container

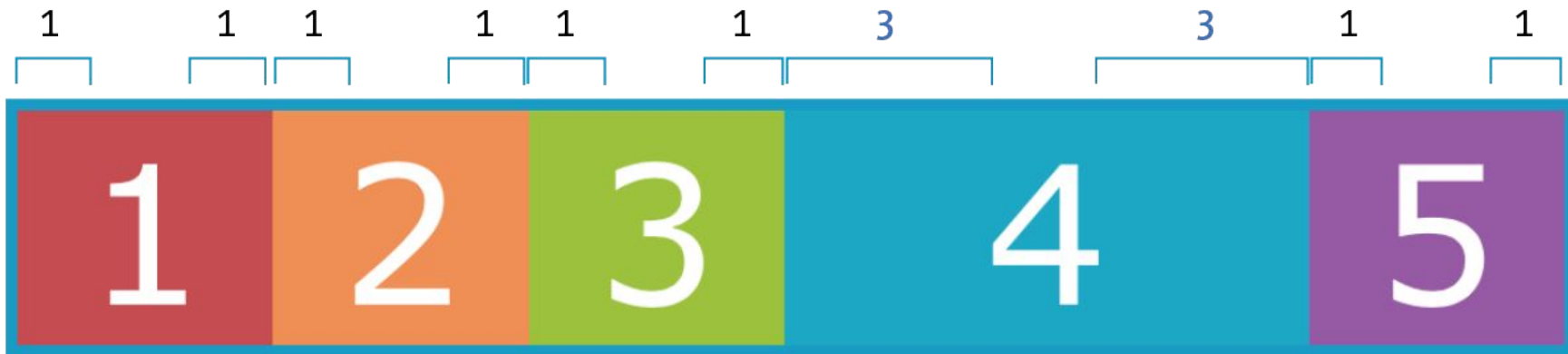
flex: 0 1 auto; (prevents expansion)



flex: 1 1 auto; (allows expansion)



# Determining How Items “Flex” in the Container



flex: 3 1 auto;

```
flex: 0 1 100px;
```



When the container is wide, the items will not grow wider than their **flex-basis** of 100 pixels because **flex-grow** is set to 0.



When the container is narrow, the items are allowed to shrink to fit (**flex-shrink**: 1).

# Changing the Order of Flex Items

## order

Values: **integer**

Default: **0**

```
.box3 {  
  order: 1;  
}
```



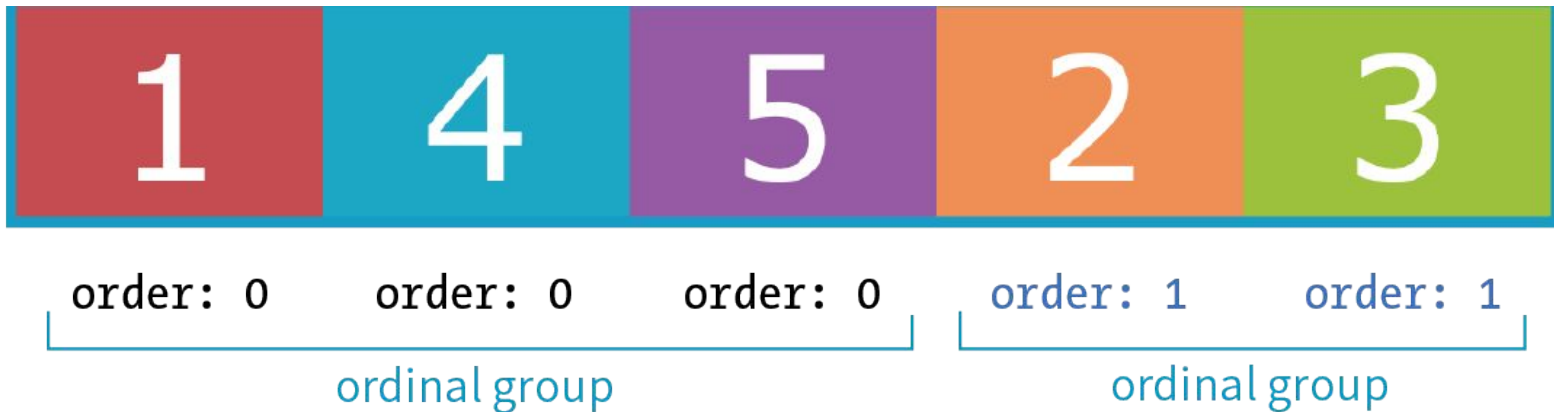
# Changing the Order of Flex Items

## order

Values: **integer**

Default: **0**

```
.box2,  
.box3 {  
  order: 1;  
}
```



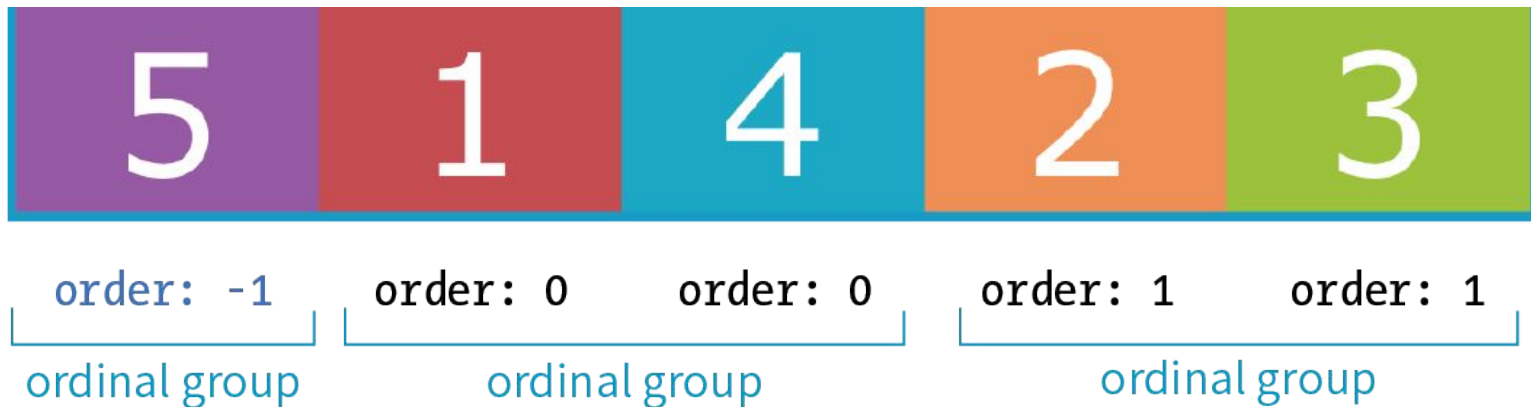
# Changing the Order of Flex Items

## order

Values: **integer**

Default: **0**

```
.box5 {  
  order: -1;  
}
```



# Applying Flexbox

How do you create the layout shown in the next slide

# Hip & Happenin' Headline

## Contact

## Where It's At

## News

Integer ornare neque enim, non imperdiet ex pellentesque sed. Sed congue, nunc ut rhoncus consectetur, diam purus venenatis lacus, vel sollicitudin nibh tortor sed nunc. Vestibulum at ante ac tortor ultricies gravida eget molestie nibh. Fusce tempor dictum lectus, id luctus sapien tristique suscipit. Cras volutpat lectus at urna vulputate pellentesque. Praesent eleifend, sapien ut finibus facilisis, orci augue elementum leo, a faucibus augue nibh ac urna. Aliquam erat volutpat. Sed ultrices tempus neque, nec iaculis orci sollicitudin id. Mauris gravida congue sapien, vitae finibus nisi condimentum id. Donec turpis metus, euismod sed luctus sit amet, semper eu purus.

```
<header>...</header>
<main>
  <article><h2>Where It's At</h2></article>
  <aside id="news"><h2>News</h2></aside>
  <aside id="contact"><h2>Contact</h2></aside>
</main>
<footer>...</footer>
```

```
main {  
  display: flex;  
}
```

```
article {  
  flex: 1 1 50%;  
  order: 2;  
}
```

```
#news {  
  flex: 1 1 25%;  
  order: 3;  
}  
  
#contact {  
  flex: 1 1 25%;  
  order: 1;  
}
```

# CSS Grid Layout

The CSS Grid Layout Module provides a system for laying out elements in rows and columns

<https://codepip.com/games/grid-garden/>

# How to use Grid Layout

Use the **display** property to turn an element into a grid container

The element's children automatically become grid items

Set up the **columns** and **rows** for the grid

You can set them up explicitly and/or provide directions for how rows and columns should get created on the fly

Assign each grid item to an area on the grid

If you don't assign them explicitly, they flow into the cells sequentially

# Grid Terminology

To make an element a grid container apply the **display: grid** property

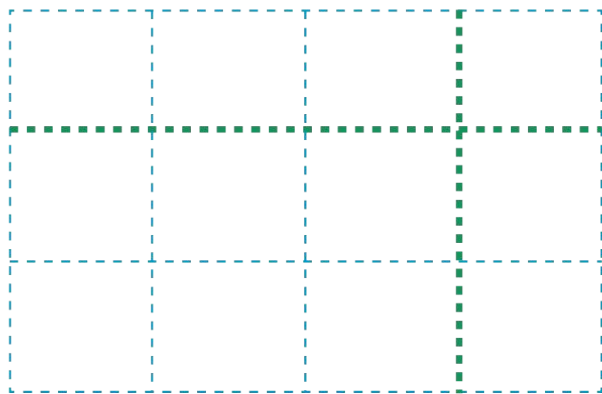
All of the elements direct child elements automatically become grid items that end up positioned in the grid

You can nest a grid inside another grid

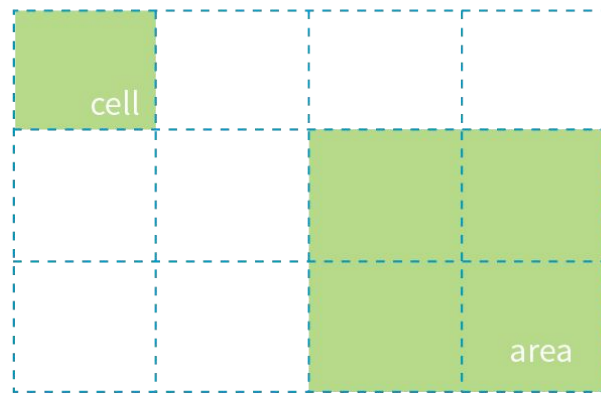
# Grid Terminology

## Grid Components

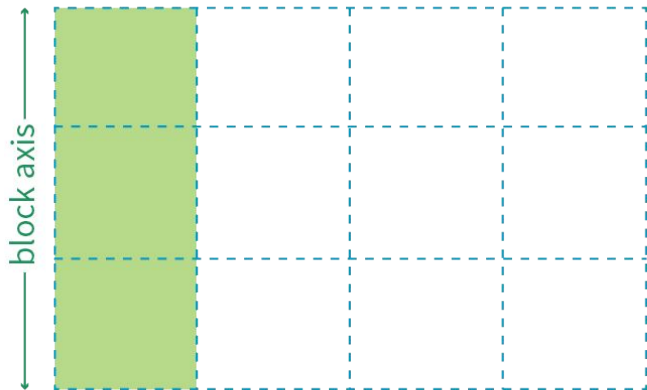
**Grid Track** is a generic name for a grid column or a grid row



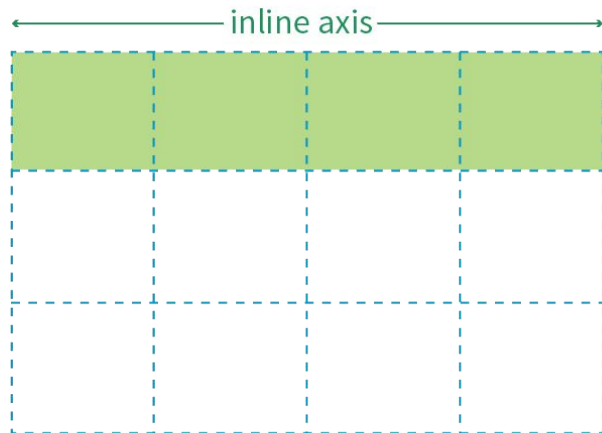
grid lines



grid cell and grid area



grid track (column)



grid track (row)

# Declaring Grid Display

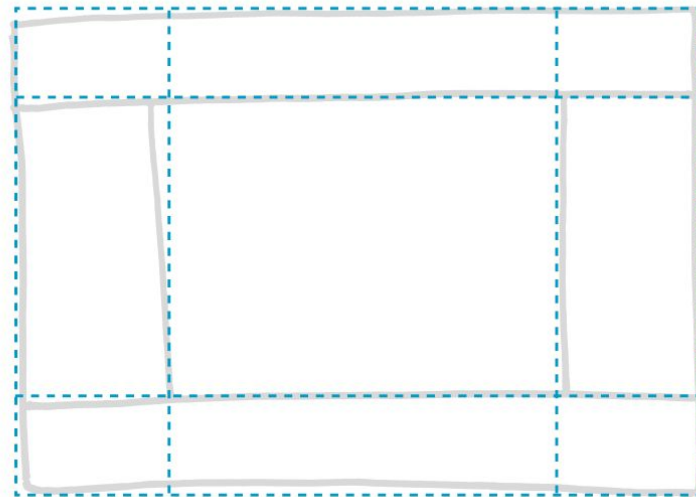
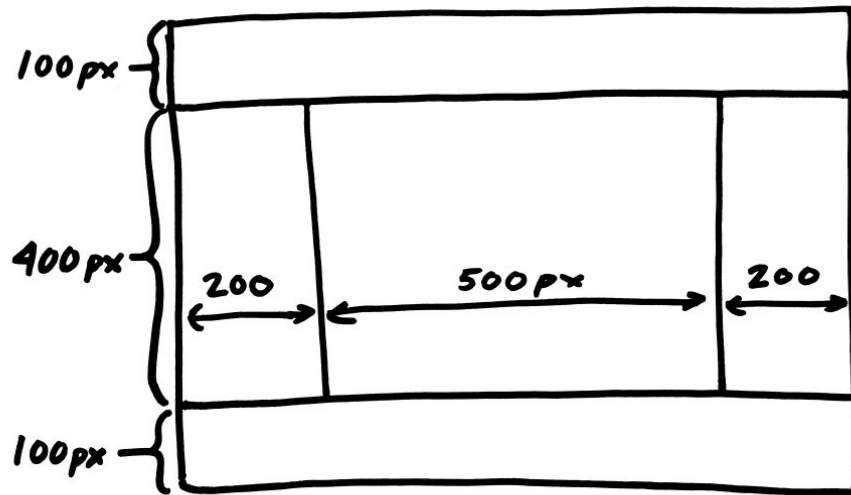
To turn an element into a grid container, set its display property to **grid** or **inline-grid**

```
#layout {  
  display: grid;  
}
```

```
<div id="layout">  
  <div id="one">One</div>  
  <div id="two">Two</div>  
  <div id="three">Three</div>  
  <div id="four">Four</div>  
  <div id="five">Five</div>  
</div>
```

# Setting Up the Grid

Let us how to setup the grid shown



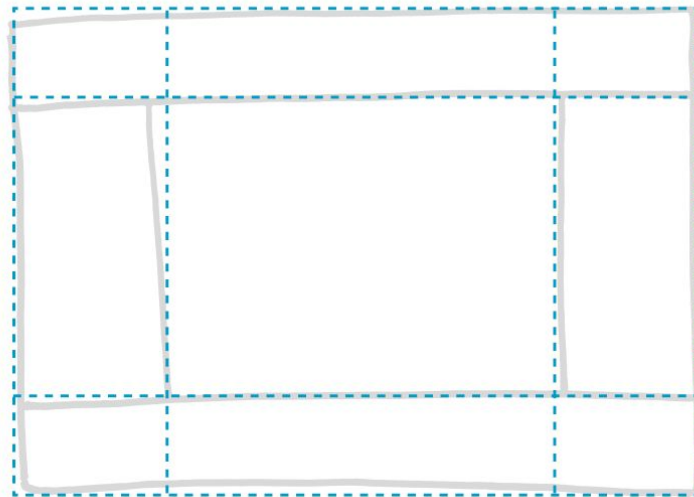
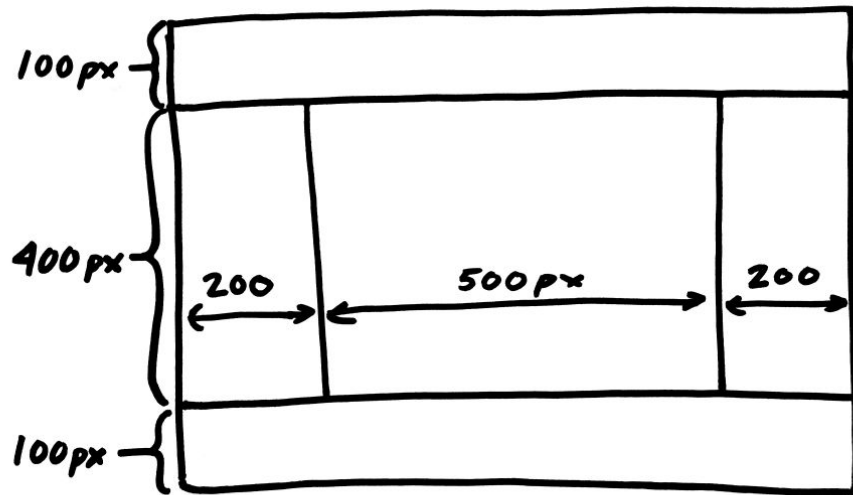
# Defining grid tracks

`grid-template-rows`

`grid-template-columns`

Values: none | list of track sizes and optional line names

Default: none



# Grid track sizes

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 200px 500px 200px;  
}
```

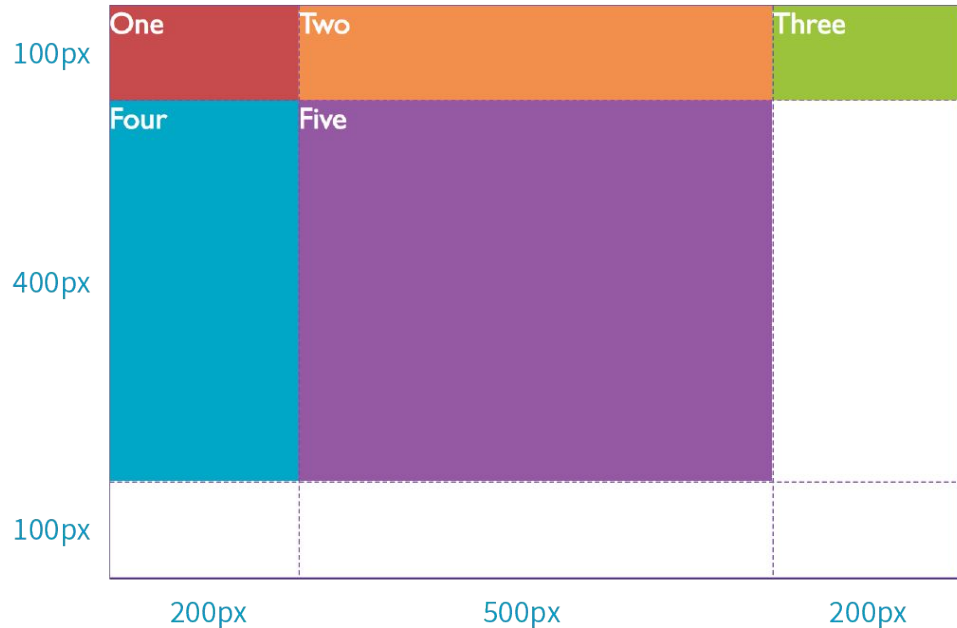
# Grid track sizes

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 200px 500px 200px;  
}
```

Browser view



Grid structure revealed with Firefox Grid Inspector



# Grid track sizes

When the browser creates a grid, it also automatically assigns each **grid line** a number that you can reference when positioning items

The **grid line at the start of the grid track is 1**, and lines are numbered sequentially

The lines are numbered from the end of tracks as well, starting with  $-1$ , and numbers count back from there ( $-2$ ,  $-3$ , etc.)

Check the next slide for an example



# Grid track sizes

You can also assign names to lines

```
#layout {  
  display: grid;  
  grid-template-rows: [header-start] 100px [content-start] 400px  
[footer-start] 100px;  
  grid-template-columns: [ads] 200px [main] 500px [links] 200px;  
}
```

The grid line at the top of the grid can now be referred to as “**header-start**,” “**1**,” or “**-4**.”

# Grid track sizes

To give a line more than one name, just include all the names in the brackets, separated by spaces

```
#layout {  
  display: grid;  
  grid-template-rows: [header-start] 100px [header-end content-start]  
400px [footer-start] 100px;  
  grid-template-columns: [ads] 200px [main] 500px [links] 200px;  
}
```

# Specifying track size values

The Grid specification provides the following values for the **grid-template-\*** properties:

|                                           |                                         |
|-------------------------------------------|-----------------------------------------|
| Lengths (such as <b>px</b> or <b>em</b> ) | <b>auto</b>                             |
| Percentage values (%)                     | <b>min-content</b> , <b>max-content</b> |
| Fractional units ( <b>fr</b> )            | <b>minmax()</b>                         |
| <b>fit-content()</b>                      |                                         |

# Specifying track size values

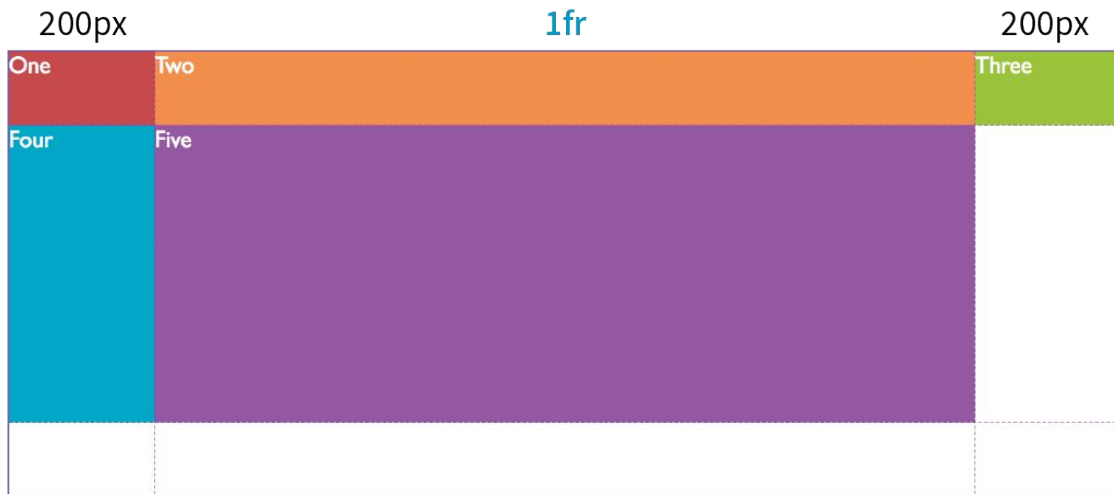
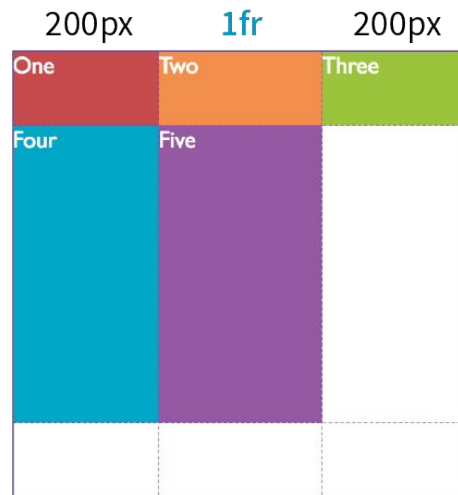
## Fractional units (flex factor)

The **Grid-specific fractional unit** (**fr**) allows developers to create track widths that expand and contract depending on available space

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 200px 1fr 200px;  
}
```

# Specifying track size values

## Fractional units (flex factor)



# Specifying track size values

## Fractional units (flex factor)

The **fr** unit is great for combining fixed and flexible track widths, but it also be used to give all the columns proportional widths

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 1fr 2fr 1fr;  
}
```

# Specifying track size values

## Minimum and maximum size range

You can constrict the size range of a track by setting its **minimum** and **maximum widths** using the **minmax()** function in place of a specific track size

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 1px minmax(15em, 45em) 1px;  
}
```

# Specifying track size values

## Content-based sizing

The **min-content**, **max-content**, and **auto** values size the track based on the size of the content within it

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 1px min-content 1px;  
}
```

Text content in cell

Look for the good in  
others and they'll see  
the good in you.

Column width set to  
`max-content`

Look for the good in others and they'll see the good in you

Column width set to  
`min-content`

Look  
for  
the  
good  
in  
others  
and  
they'll  
see  
the  
good  
in  
you.

# Specifying track size values

## Repeating track sizes

A grid that has 10 columns with alternating column widths

```
grid-template-columns: 20px 1fr 20px 1fr 20px 1fr 20px  
1fr 20px 1fr;
```

Shortcut

```
grid-template-columns: repeat(5, 20px 1fr);
```

# Specifying track size values

## **auto-fill** and **auto-fit**

You can also let the browser figure it out itself based on the available space by using the **auto-fill** and **auto-fit** values instead of an integer

```
grid-template-columns: grid-template-rows: repeat(auto-fill, 10em);
```

# Defining grid areas

To assign names to grid areas, use the **grid-template-areas** property

**grid-template-areas**

Values: `none` | `series of area names`

Default: `none`

# Defining Grid Areas

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 200px 1fr 200px;  
  grid-template-areas:  
    "header  header  header"  
    "ads      main    links"  
    "footer  footer  footer";  
}
```

# Defining Grid Areas

|          |        |         |
|----------|--------|---------|
| "header" | header | header" |
| "ads"    | main   | links"  |
| "footer" | footer | footer" |

|     |        |       |
|-----|--------|-------|
|     | header |       |
| ads | main   | links |
|     | footer |       |

# Defining Grid Areas

If there are three columns in the grid, there must be three names provided for each row

If you want to leave a cell unnamed, type one or more periods (.) in its place as a space holder so that every cell is still accounted for

# The grid shorthand property

Use the **grid** shorthand property to set values for **grid-template-rows**, **grid-template-columns**, and **grid-template-areas** with one style rule

## **grid**

Values: none | row info / column info

Default: none

# The grid shorthand property

## grid

Values: none | row info / column info

Default: none

```
#layout {  
  display: grid;  
  grid: 100px 400px 100px / 200px 1fr 200px;  
}
```

# The grid shorthand property

To include custom line names, add the names in brackets around their respective tracks

Area names should appear with the row information as shown below

```
[start line name] "area names" <track size> [end line name]
```

# The grid shorthand property

The expanded equivalent of the following is shown in the next slide

```
#layout {  
  display: grid;  
  grid:  
    [header-start] "header  header  header" 100px  
    [content-start] "ads      main    links" 400px  
    [footer-start] "footer  footer  footer" 100px  
  / [ads] 200px [main] 1fr [links] 200px;  
}
```

# The grid shorthand property

The expanded equivalent of the example is shown in the previous slide

```
#layout {  
  display: grid;  
  grid-template-rows: [header-start] 100px [content-start] 400px  
[footer-start] 100px;  
  grid-template-columns: [ads] 200px [main] 1fr [links] 200px;  
  grid-template-areas:  
    "header  header  header"  
    "ads      main   links"  
    "footer  footer  footer";  
}
```

# Placing Grid Items

Without any explicit placement instruction, grid items flow into the available grid cells sequentially

# Positioning using lines

One method for describing a grid item's location on the grid is to **specify the four lines bordering the target grid area** with four properties that specify the **start and end row lines** and the **start and end column lines**

# Positioning using lines

`grid-row-start`

`grid-row-end`

`grid-column-start`

`grid-column-end`

Values: `auto` | `grid line` | `span number` | `span 'line name'` |  
`number 'line name'`

Default: `auto`

# Positioning using lines

```
#one {  
  grid-row-start: 1;  
  grid-row-end: 2;  
  grid-column-start: 1;  
  grid-column-end: 4;  
}
```

# Positioning using lines

Using name names is also possible

```
#one {  
  grid-row-start: header-start;  
  grid-row-end: header-end;  
}
```

# Positioning using lines

You can use the `span` keyword to specify how many tracks to span over.

```
#one {  
  grid-column-start: 1;  
  grid-column-end: span 3;  
}
```

```
#one {  
  grid-column-start: span 3;  
  grid-column-end: -1;  
}
```

# Positioning using lines

Shorthand `grid-row` and `grid-column` properties

`grid-row`

`grid-column`

Values: start line / end line

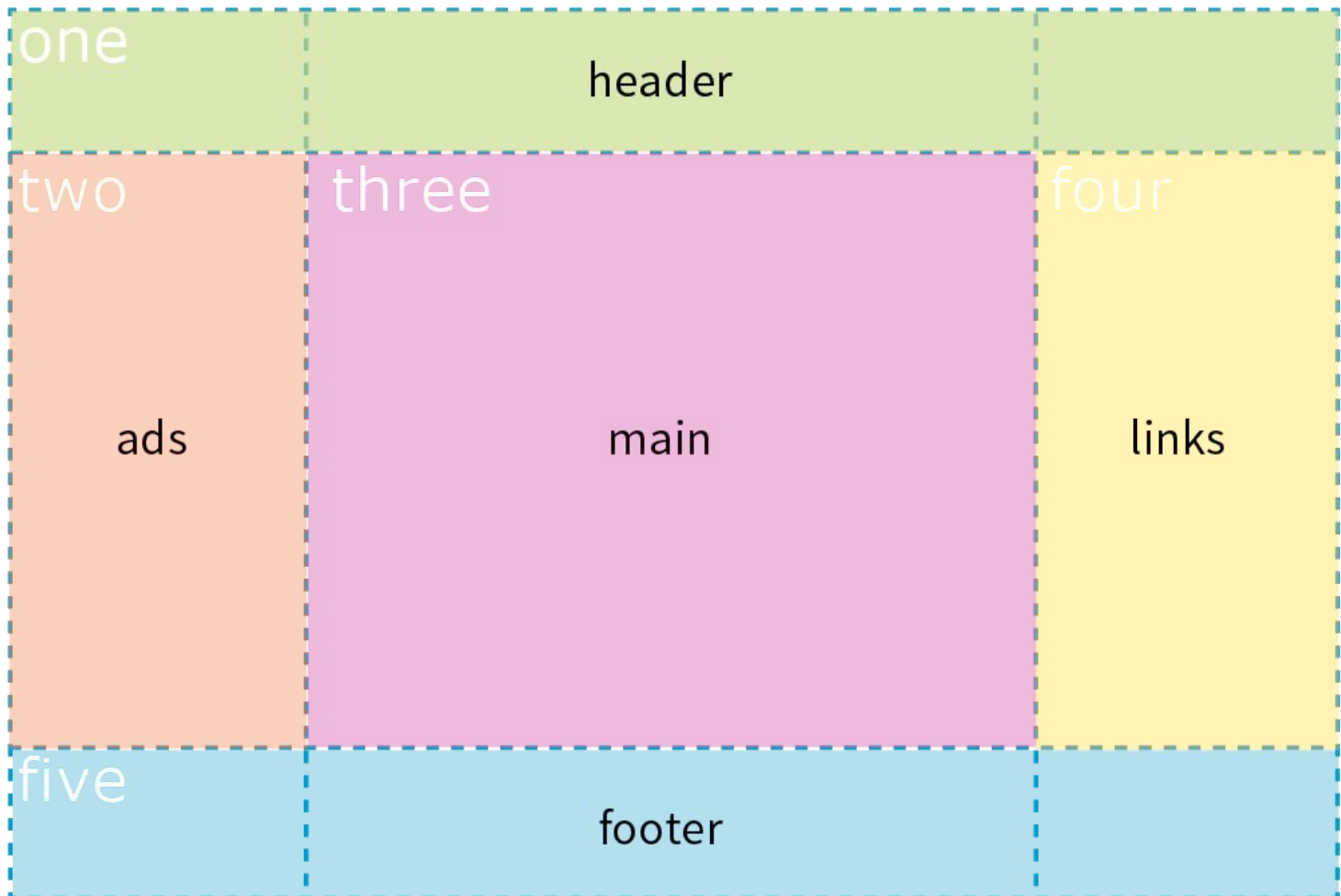
```
#one {  
  grid-row: 1 / 2;  
  grid-column: 1 / span 3;  
}
```

# Positioning by area

## grid-area

Values: area name | 1 to 4 line identifiers

```
#one {grid-area: header;}  
#two {grid-area: ads;}  
#three {grid-area: main;}  
#four {grid-area: links;}  
#five {grid-area: footer;}
```



# Positioning by area

You can also use `grid-area` to provide a list of four grid lines that define an area, separated by slashes

The order in which they appear is “row-start,” “column-start,” “row-end,” “column-end” (counterclockwise from the top)

```
#one {  
  grid-area: 1 / 1 / 2 / span 3;  
  /* row-start / column-start / row-end / column-end */  
}
```

# Implicit Grid Behavior

Without explicit placement instructions, grid items flow into the grid sequentially

Creating a named area implicitly generated grid lines with the “-**start**” and “-**end**” suffixes

if you place an item outside a defined grid, the browser automatically generates tracks in the grid to accommodate it

if you simply have more items than there are cells or areas, the browser generates more tracks until all the items are placed

# Implicit Grid Behavior

By default, any row or column automatically added to a grid will have the size auto, sized just large enough to accommodate the height or width of the contents

If you want to give implicit rows and columns specific dimensions, use the **grid-auto-\*** properties

# Implicit Grid Behavior

**grid-auto-rows**

**grid-auto-columns**

Values: `list of track sizes`

Default: `auto`

The **grid-auto-rows** and **grid-auto-columns** properties provide one or more track sizes for automatically generated tracks and apply to the grid container

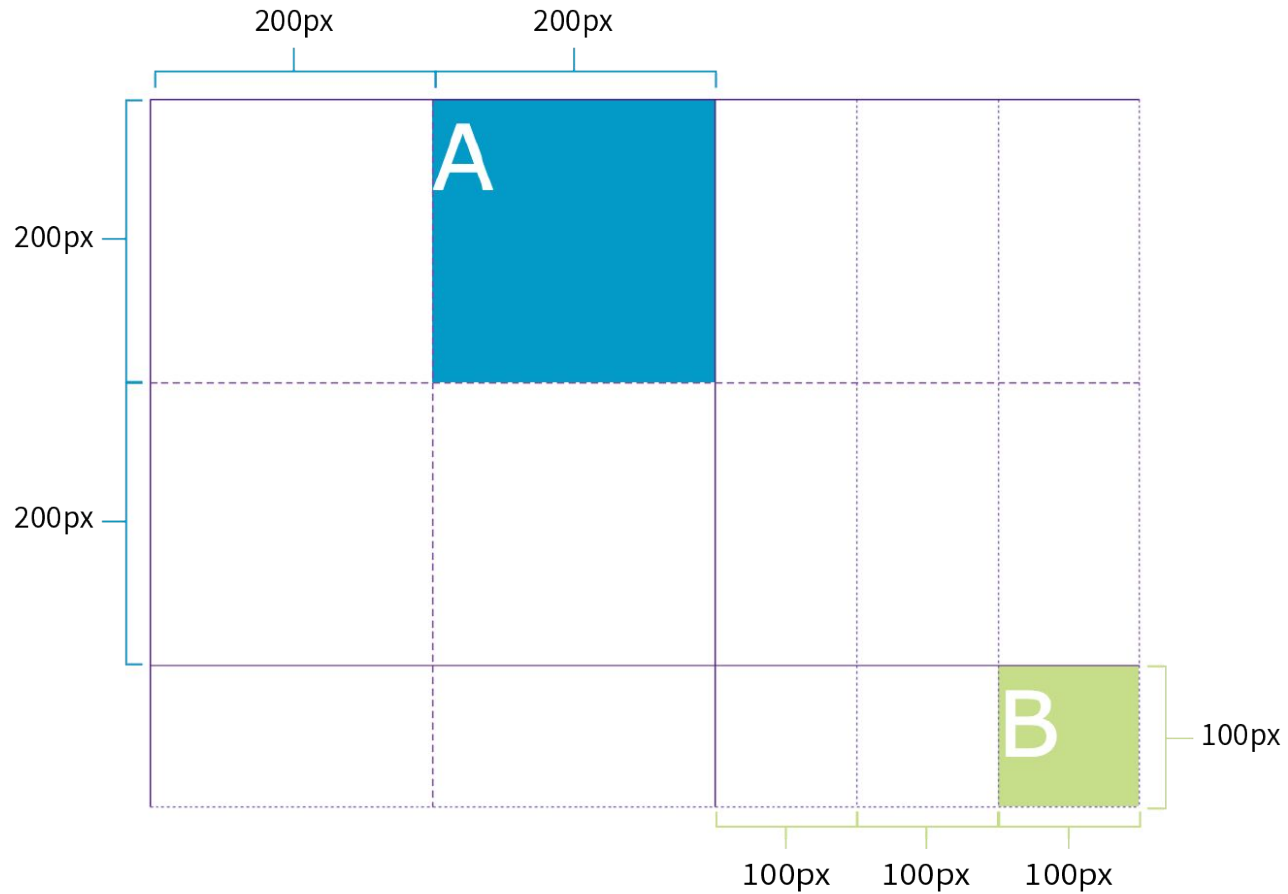
# Implicit Grid Behavior

```
#littlegrid {  
  display: grid;  
  grid-template-columns: 200px 200px;  
  grid-template-rows: 200px 200px;  
  grid-auto-columns: 100px;  
  grid-auto-rows: 100px;  
}
```

```
#A {  
  grid-row: 1 / 2;  
  grid-column: 2 / 3;  
}
```

```
#B {  
  grid-row: 3 / 4;  
  grid-column: 5 / 6;  
}
```

The grid has two explicitly defined rows and columns at 200 pixels wide each.



Rows and column tracks are added automatically as needed. They are sized as specified by `grid-auto-rows` and `grid-auto-columns` (100 pixels).

# Implicit Grid Behavior

A more common use of auto-generated tracks is to tile images, product listings, and the like into columns, letting rows be created as needed

The following styles set up a grid with explicit columns and as many 200px-high rows as needed

```
grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));  
grid-auto-rows: 200px;
```

# Flow direction and density

## **grid-auto-flow**

Values: `row` or `column` | `dense` (optional)

Default: `row`

# Flow direction and density

Use **grid-auto-flow** to specify whether you'd like items to flow in by row or column

The default flow follows the writing direction of the document (left-to-right and top-to-bottom for English and other left-to-right languages)

```
#listings {  
  display: grid;  
  grid-auto-flow: column;  
}
```

# Flow direction and density

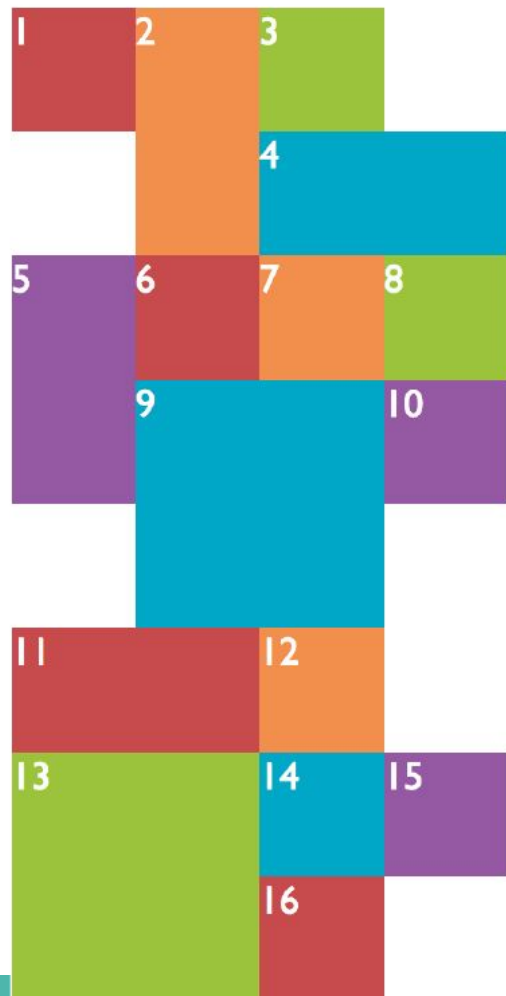
By default, items are placed in the first area in which they fit

Cells that are too small to accommodate the content will be skipped over until a cell large enough is found for placement

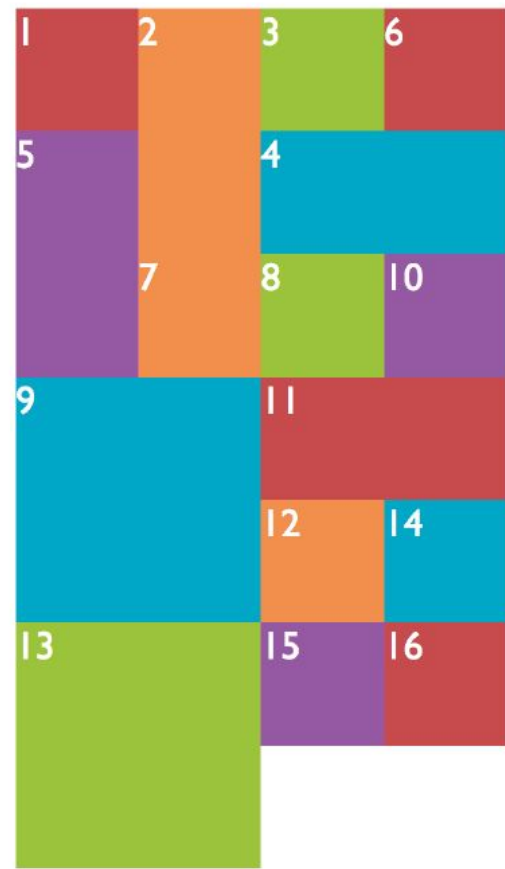
The **dense** keyword for the **grid-auto-flow** property make grid items fill the grid as densely as possible, allowing the items to appear out of sequence in order to fill the available space

```
#listings {  
  display: grid;  
  grid-auto-flow: dense rows;  
}
```

Default flow pattern



Dense flow pattern



# Spacing and Alignment

You can add space between tracks and adjust alignment of the grid and its items

# Spacing between tracks (gutters)

**grid-row-gap**

**grid-column-gap**

Values: `length` (must not be negative)

Default: 0

# Spacing between tracks (gutters)

## **grid-gap**

Values: `grid-row-gap` `grid-column-gap`

Default: 0 0

```
div#container {  
  border: 2px solid gray;  
  display: grid;  
  grid: repeat(4, 150px) / repeat(4, 1fr);  
  grid-gap: 20px 50px;  
}
```



```
grid-row-gap: 20px;  
grid-column-gap: 50px;
```

# Aligning individual items

## **justify-self**

Values: start | end | center | left | right | self-start |  
self-end | stretch | normal | auto

Default: auto (looks at the value for justify-items, which defaults to normal)

specifies alignment on the horizontal (inline) axis

# Aligning individual items

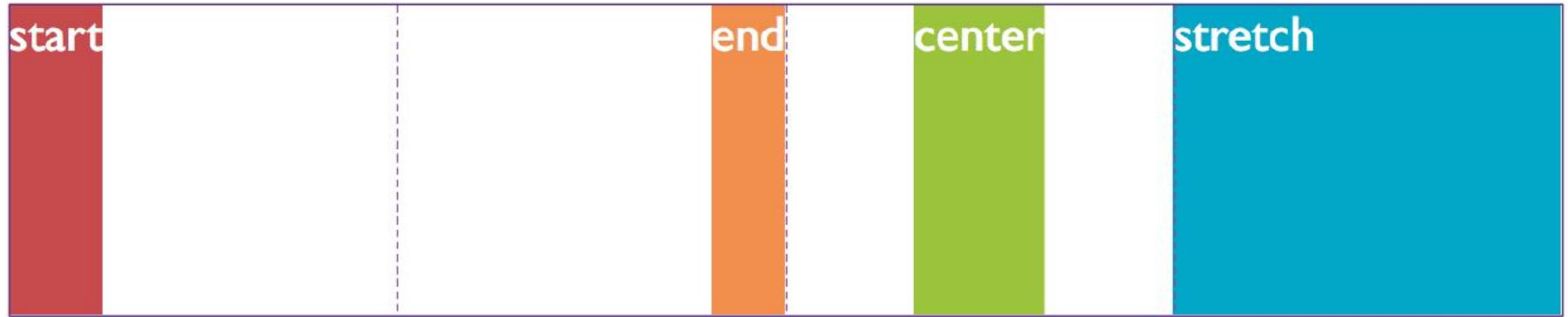
## **align-self**

Values: `start` | `end` | `center` | `left` | `right` | `self-start` |  
`self-end` | `stretch` | `normal` | `auto`

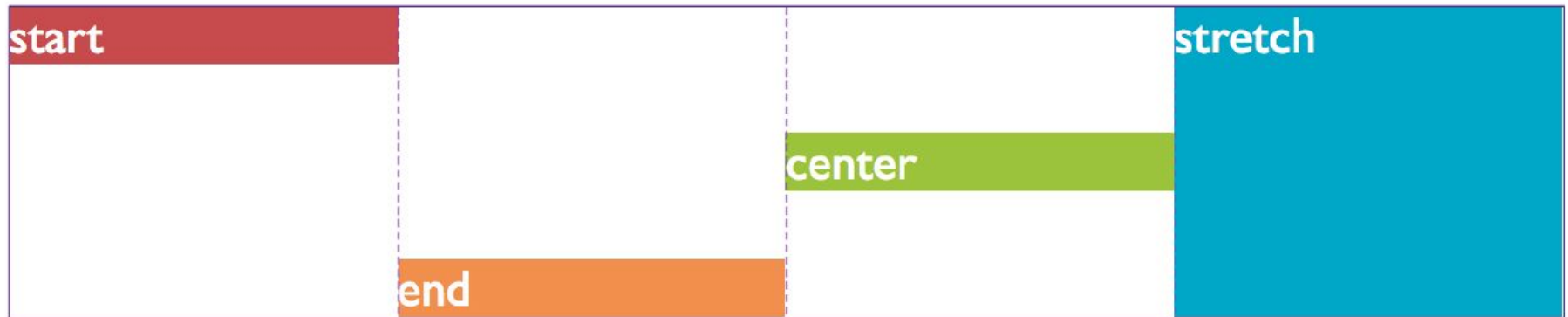
Default: `auto` (looks at the value for `align-items`)

specifies alignment on the vertical (block) axis

## justify-self



## align-self



# Aligning all the items in a grid

## **justify-items**

Values: `start` | `end` | `center` | `left` | `right` | `self-start` |  
`self-end` | `stretch` | `normal`

Default: `normal` (stretch for non-replaced elements; start for replaced elements)

# Aligning all the items in a grid

## **align-items**

Values: `start` | `end` | `center` | `left` | `right` | `self-start` |  
`self-end` | `stretch` | `normal`

Default: `normal` (stretch for non-replaced elements; start for replaced elements)

# Aligning tracks in the grid container

There may be instances in which the tracks of your grid do not fill the entire area of their grid container

For example, if you've specified track widths and heights in specific pixel measurements

You can decide how the browser should handle leftover space within the container by using the

`justify-content` (horizontal/inline axis) and

`align-content` (vertical/block axis) properties

# Aligning tracks in the grid container

## **justify-content**

Values: `start` | `end` | `left` | `right` | `center` | `stretch` |  
`space-around` | `space-between` | `space-evenly`

Default: `start`

# Aligning tracks in the grid container

## **align-content**

Values: `start` | `end` | `left` | `right` | `center` | `stretch` |  
`space-around` | `space-between` | `space-evenly`

Default: `start`

justify-content:

start

|      |     |        |        |  |
|------|-----|--------|--------|--|
| One  | Two | Three  | Four   |  |
| Five | Six | Seven  | Eight  |  |
| Nine | Ten | Eleven | Twelve |  |

end

|  |  |      |     |        |        |
|--|--|------|-----|--------|--------|
|  |  | One  | Two | Three  | Four   |
|  |  | Five | Six | Seven  | Eight  |
|  |  | Nine | Ten | Eleven | Twelve |

center

|  |      |     |        |        |
|--|------|-----|--------|--------|
|  | One  | Two | Three  | Four   |
|  | Five | Six | Seven  | Eight  |
|  | Nine | Ten | Eleven | Twelve |

space-around

|      |     |        |        |
|------|-----|--------|--------|
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

space-between

|      |     |        |        |
|------|-----|--------|--------|
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

space-evenly

|      |     |        |        |
|------|-----|--------|--------|
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

align-content:

start

|      |     |        |        |
|------|-----|--------|--------|
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

end

|      |     |        |        |
|------|-----|--------|--------|
|      |     |        |        |
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

center

|      |     |        |        |
|------|-----|--------|--------|
|      |     |        |        |
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

space-around

|      |     |        |        |
|------|-----|--------|--------|
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

space-between

|      |     |        |        |
|------|-----|--------|--------|
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

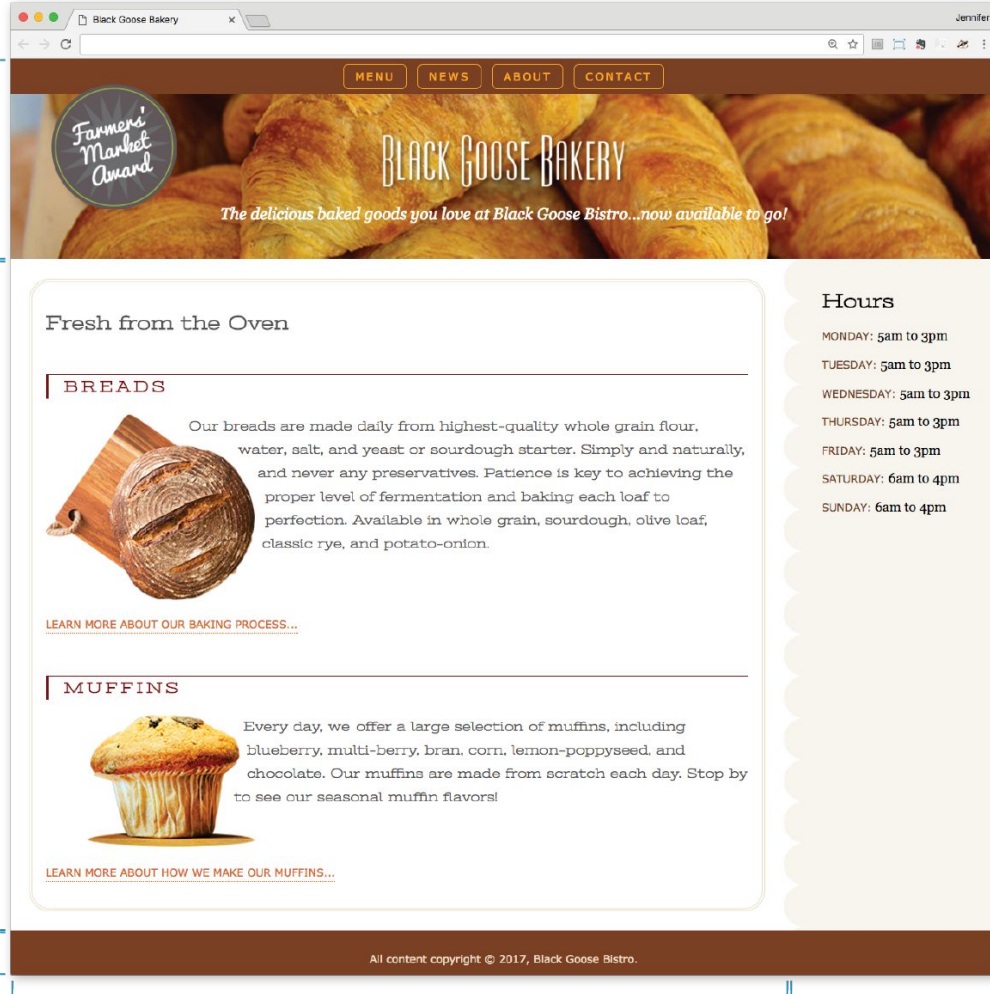
space-evenly

|      |     |        |        |
|------|-----|--------|--------|
| One  | Two | Three  | Four   |
| Five | Six | Seven  | Eight  |
| Nine | Ten | Eleven | Twelve |

# A Grid Layout Example <sup>150px</sup>

min-content

5em



minimum 25em, maximum 1fr

16em

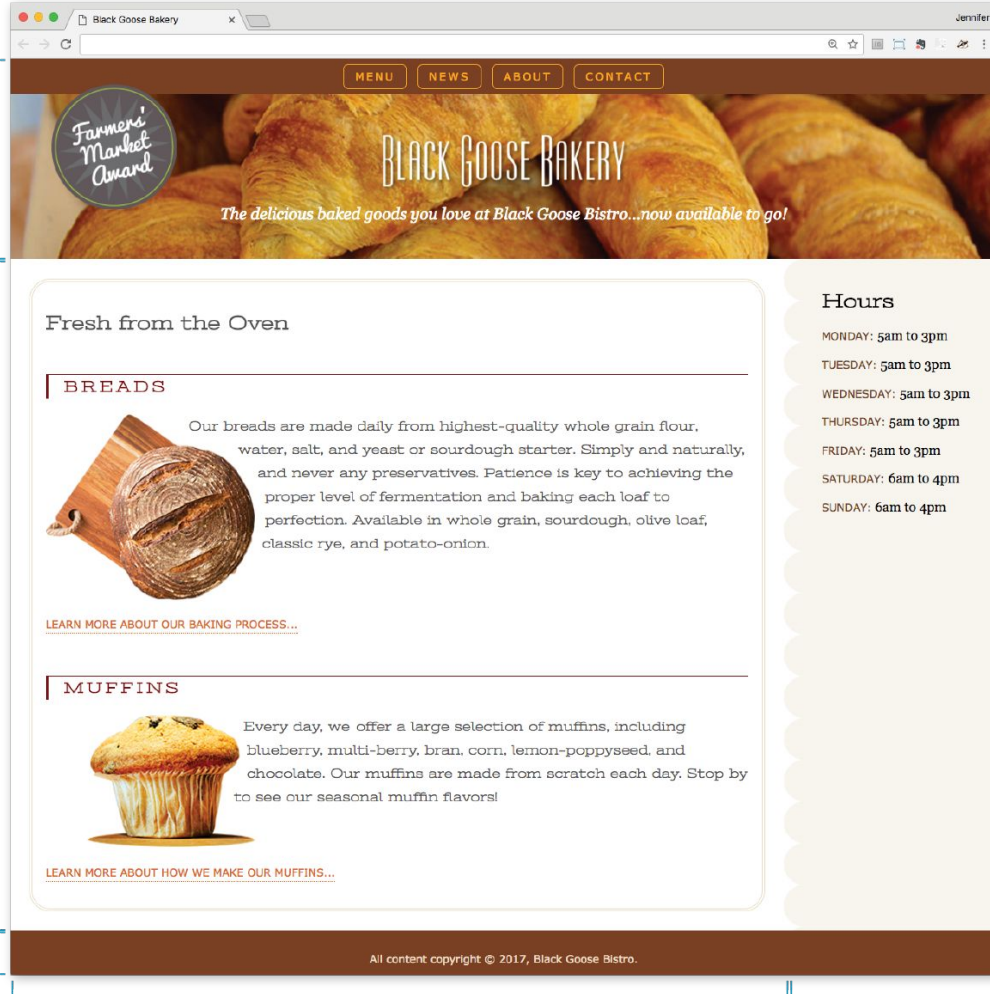
# A Grid Layout Example <sup>150px</sup>

## The HTML

```
<div id="container">
  <header>...</header>
  <main>...</main>
  <aside>...</aside>
  <footer>...</footer>
</div>
```

min-content

5em



minimum 25em, maximum 1fr

16em

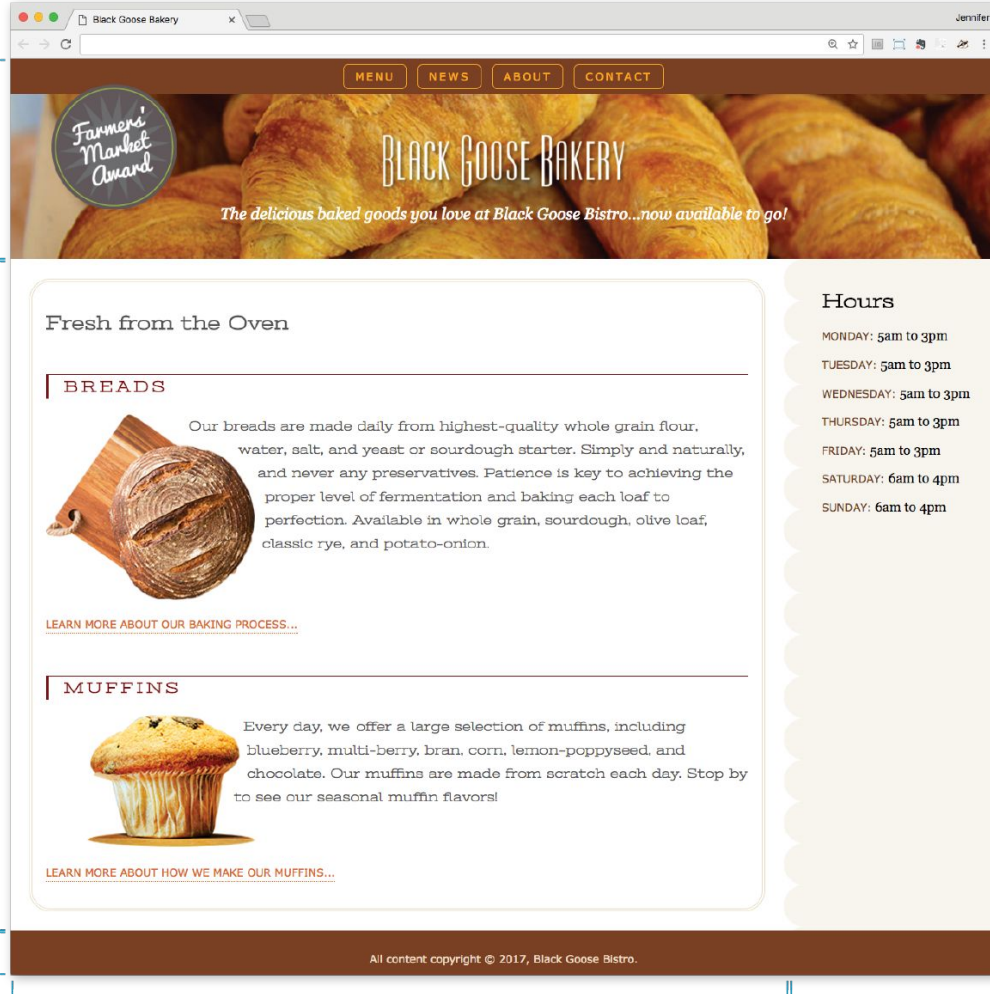
# A Grid Layout Example <sup>150px</sup>

## Grid Container

```
#container {  
  display: grid;  
}
```

min-content

5em



minimum 25em, maximum 1fr

16em

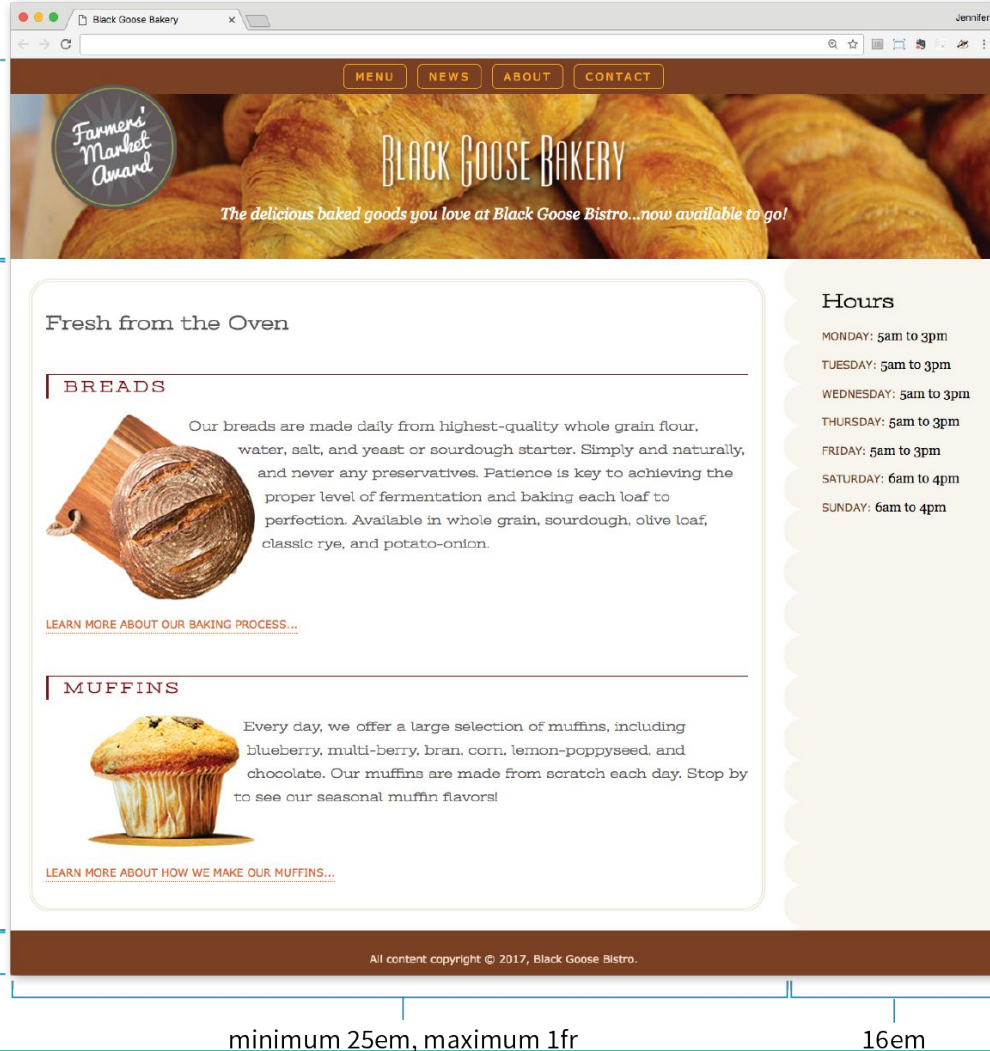
# A Grid Layout Example <sup>150px</sup>

## The Rows

```
#container {  
  display: grid;  
  grid-template-rows: auto  
  auto 5em;  
}
```

min-content

5em



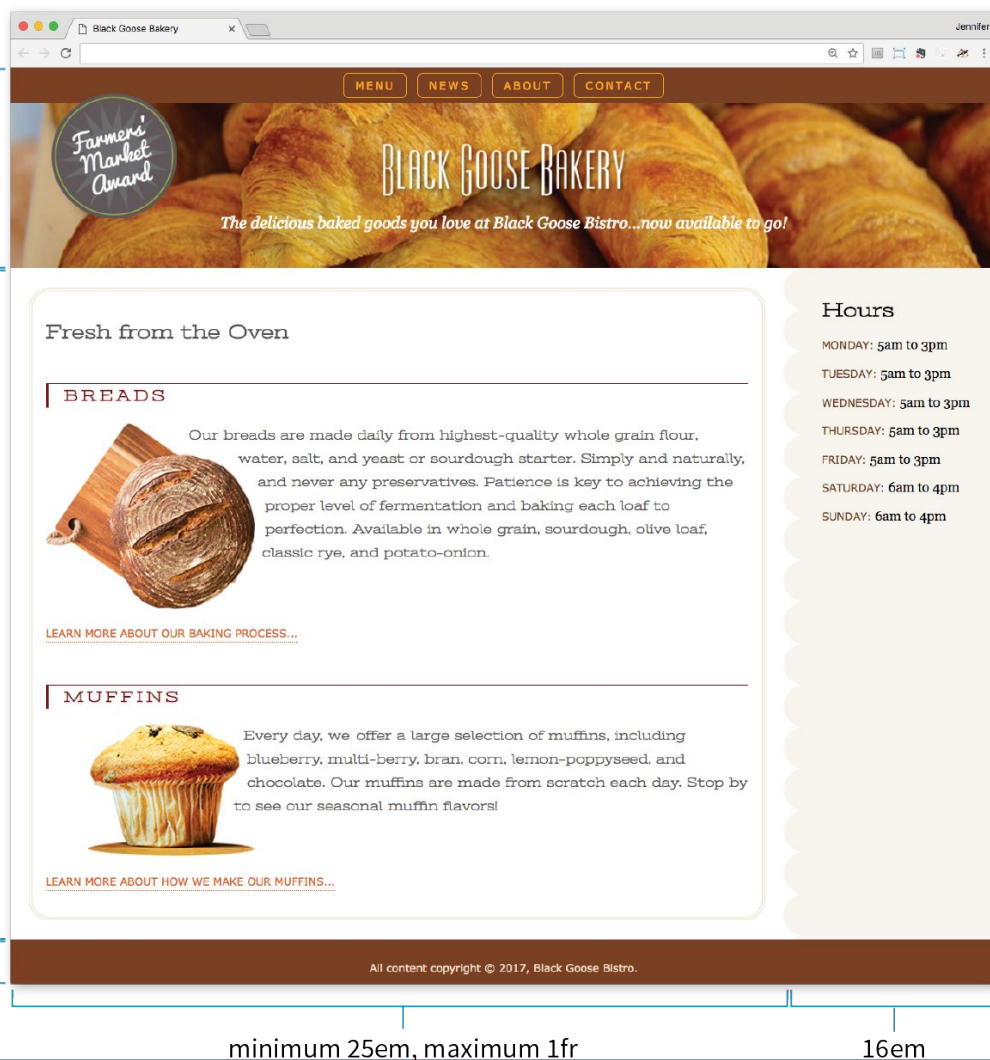
# A Grid Layout Example <sup>150px</sup>

## The Columns

```
#container {  
  display: grid;  
  grid-template-rows: auto  
  auto 5em;  
  grid-template-columns:  
  minmax(25em, 1fr) 16em;  
}
```

min-content

5em



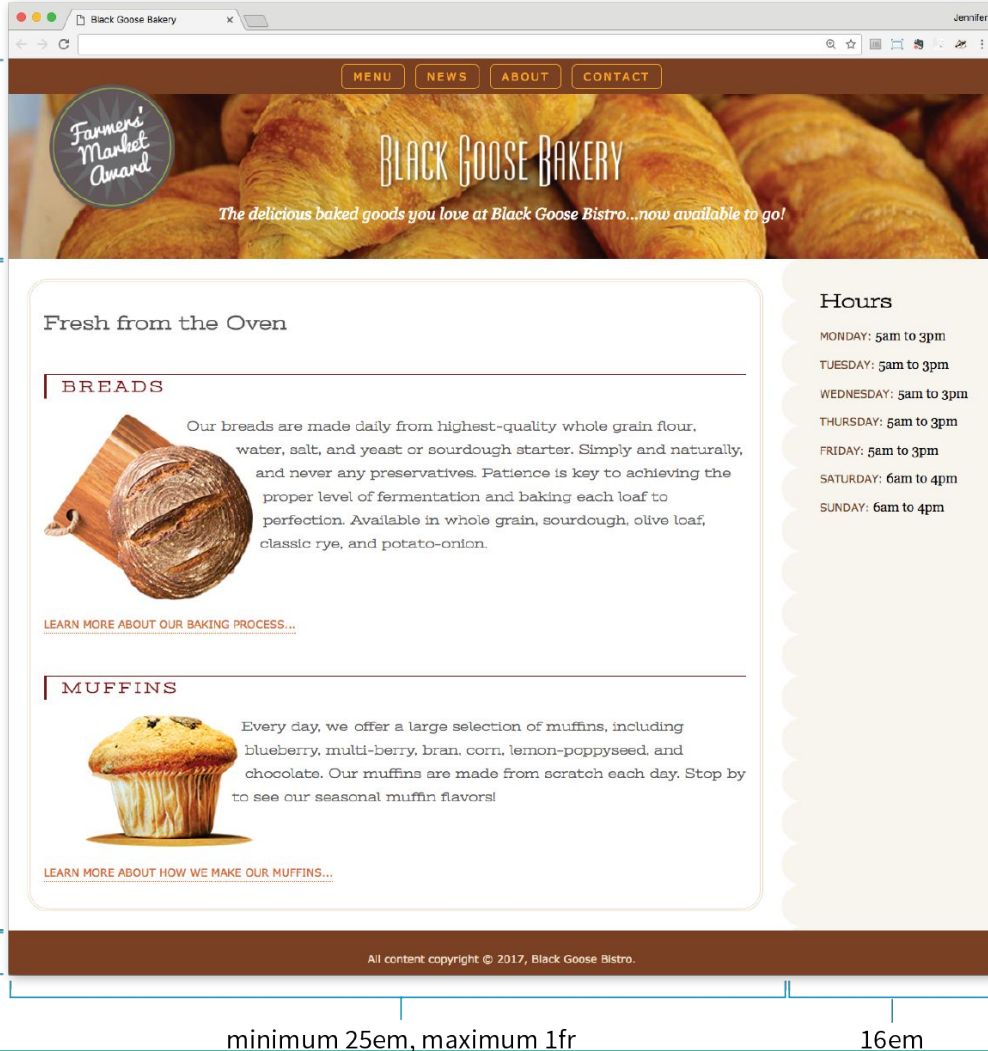
# A Grid Layout Example <sup>150px</sup>

## Naming Areas

```
#container {  
  ...  
  grid-template-areas:  
    "banner banner"  
    "main    hours"  
    "footer footer";  
}
```

min-content

5em



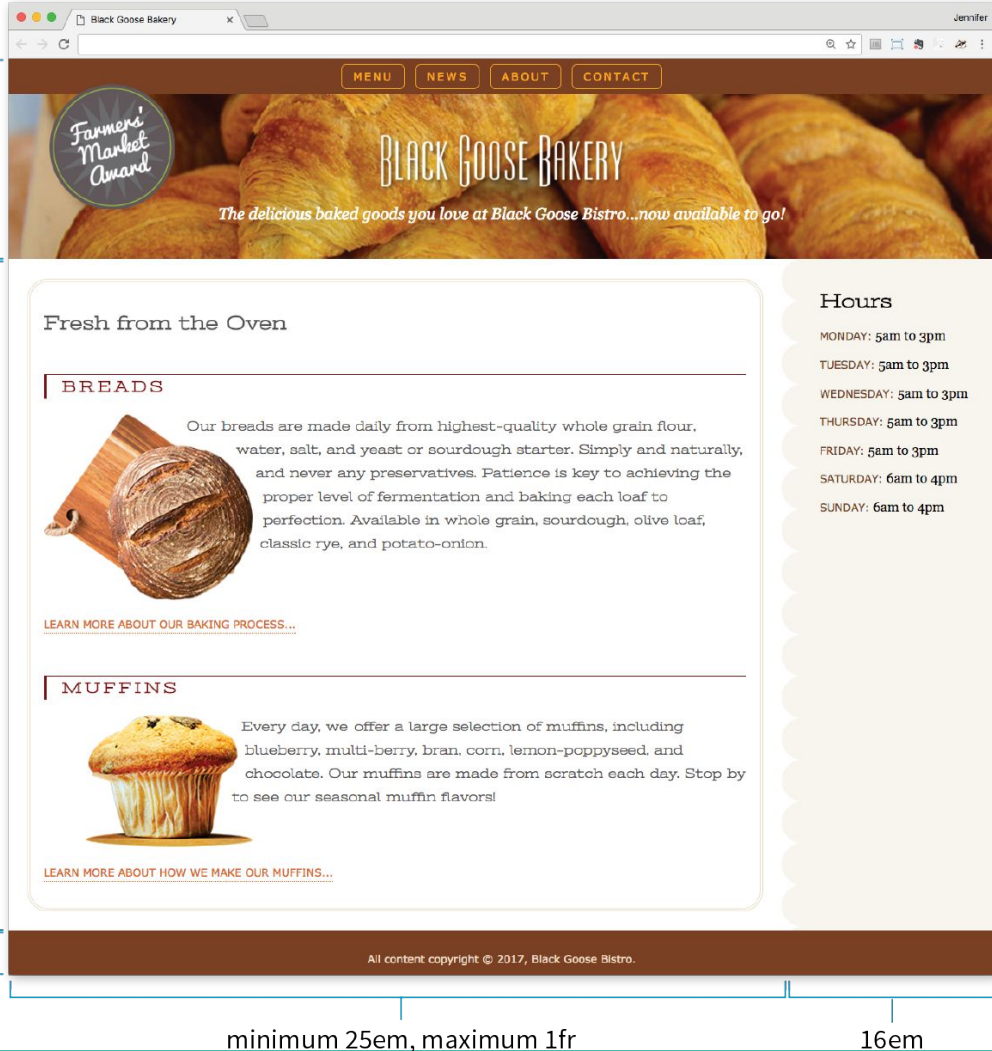
# A Grid Layout Example <sup>150px</sup>

## Placing header Grid Items

```
header {  
  grid-area: banner;  
}
```

min-content

5em



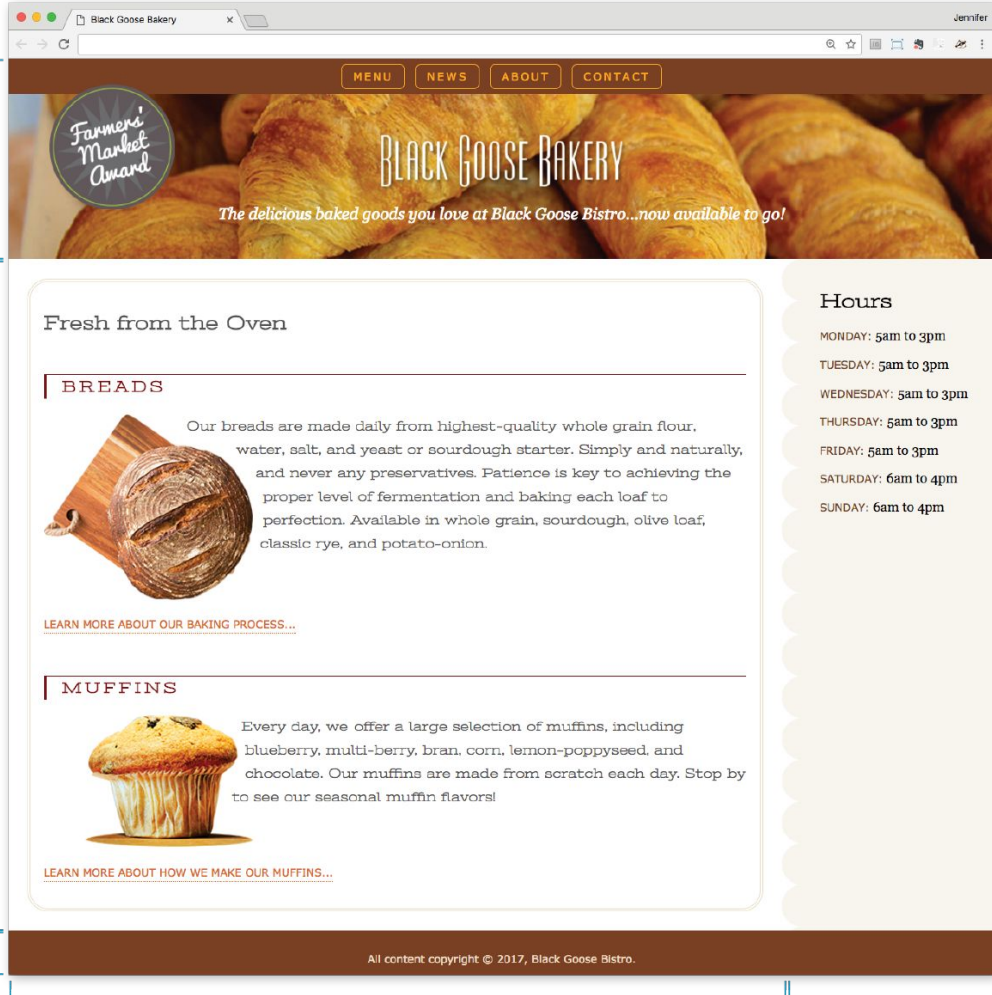
# A Grid Layout Example <sup>150px</sup>

## Placing **main** Grid Items

```
main {  
  grid-area: main;  
}
```

min-content

5em



minimum 25em, maximum 1fr

16em

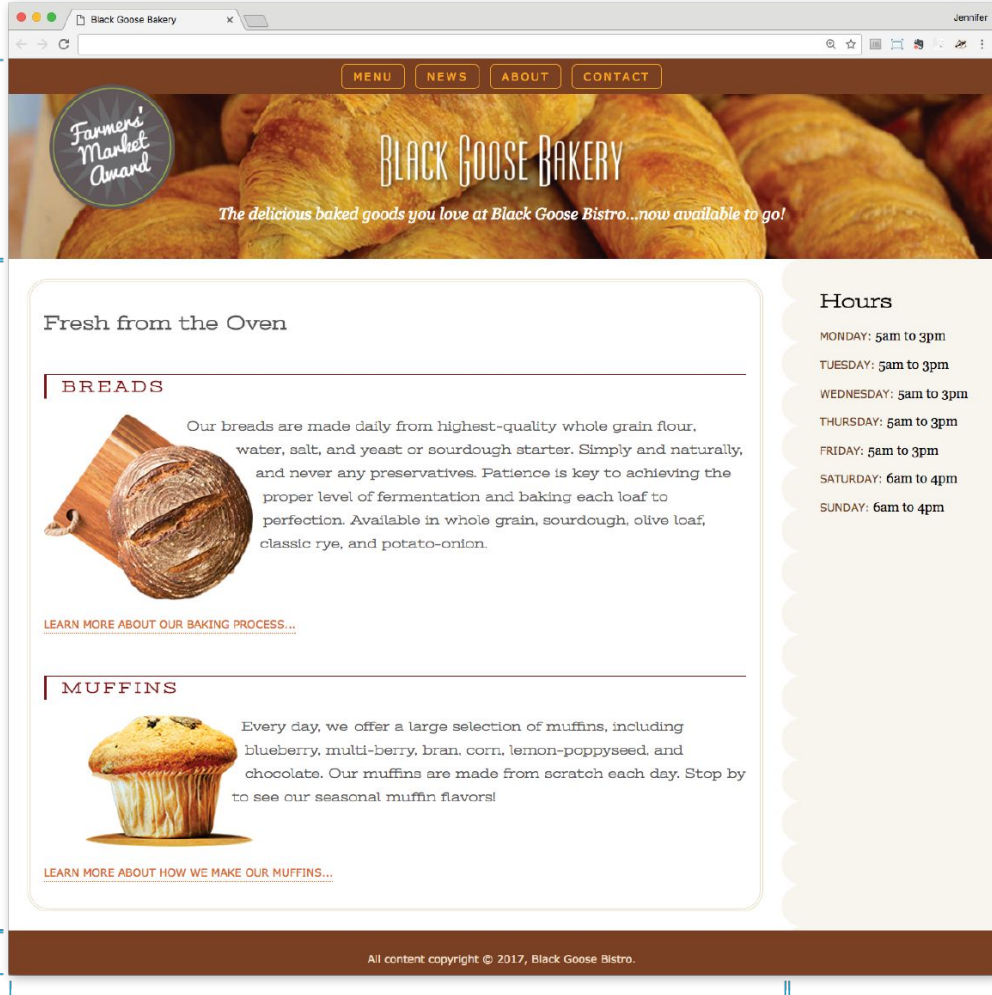
# A Grid Layout Example <sup>150px</sup>

## Placing **aside** Grid Items

```
aside {  
  grid-area: hours;  
}
```

min-content

5em



minimum 25em, maximum 1fr

16em

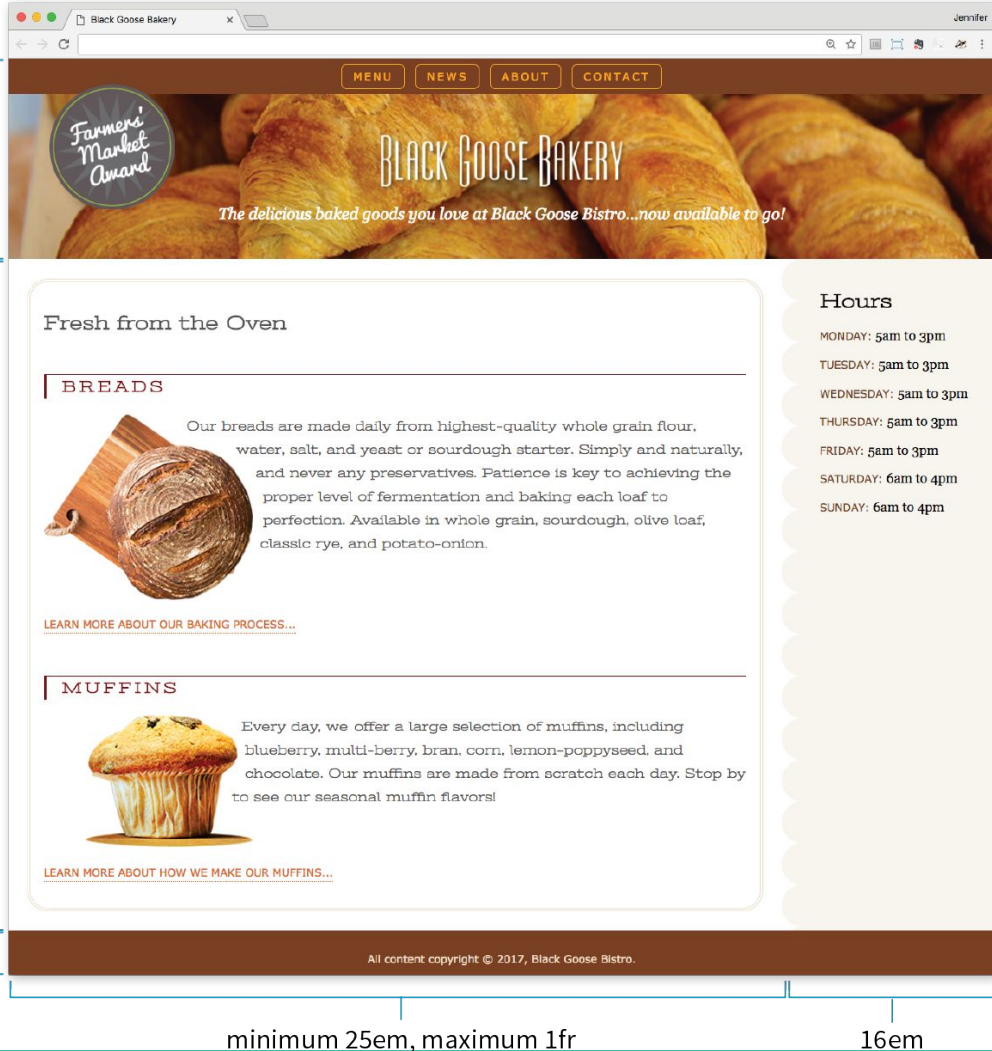
# A Grid Layout Example <sup>150px</sup>

## Placing **footer** Grid Items

```
footer {  
  grid-area: footer;  
}
```

min-content

5em



# Must Visit References

[A Complete Guide to Flexbox](#)

[A Complete Guide to Grid](#)